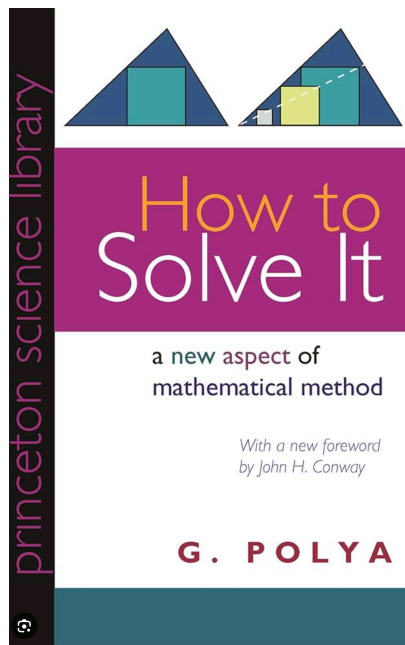


Polya



uPCB

Understand : #2259

github.com/google-deepmind/mujoco/issues/2259

google-deepmind / mujoco

[MJX] jax.lax.while_loop in solver.py prevents computation of backward gradients #2259

Open #3264 (+2)

EGalahad opened on Nov 30, 2024

The feature, motivation and pitch

Problem

The solver's `jax.lax.while_loop` implementation prevents gradient computation through the environment step during gradient based trajectory optimization. This occurs in the [solver implementation](#) when iterations > 1.

Error encountered with `jax.jit` compiled grad function:

```
ValueError: Reverse-mode differentiation does not work for lax.while_loop or lax.fori_loop with dynamic sta...
```

Current workaround of using `opt.iteration=1` leads to potentially inaccurate simulation and gradients.

Proposed Solution

Add an option to set a fixed iteration count (e.g., 4) that would be compatible with reverse-mode differentiation using either `lax.scan` or `lax.fori_loop` with static bounds.

Alternatives

No response

Additional context

No response

5

EGalahad added **enhancement** on Nov 30, 2024

saran-t assigned **erikfrey** and **btaba** on Dec 4, 2024

erikfrey added **good first issue** on Dec 4, 2024

Assignees

- btaba
- erikfrey

Labels

- MJX
- enhancement
- good first issue

Type

No type

Projects

No projects

Milestone

No milestone

Relationships

None yet

Development

- feat(mjx): Make solver differentiable and fix name collisions
google-deepmind/mujoco
- [MJX] Fix gradient computation by replacing `jax.lax.while_loop` with `_while_loop_scan`
theshloksschauhan/mujoco
- [MJX] Use scan-based loop in solver to enable reverse-mode autodiff (#2259)
google-deepmind/mujoco

Notifications

Customize

Subscribe

You're not receiving notifications from this thread.

Xhrota Umar Alkolon

JAX . LAX

Just Autograd X ~~erated~~ Linear Algebra

Plans

:

includes mujoco
mjx
model
etc

```
def update_constraint(m: Model, d: Data, ctx: Context) -> Context:
    """
    Update the constraints of the model.
    """
    if not isinstance(m, mujoco.mjx.Model) or not isinstance(d, mujoco.mjx.Data):
        raise ValueError("update_constraint requires MJX backend implementation.")

    # no constraints are always active, nf are conditionally active, others are
    # non-negative constraints.
    active = (ctx.lref < 0) & (~d._impl.ne + d._impl.nf).set(True)

    # force, mass, cost = 3p.zeros(d._impl.nofc, 0.0)
    def _f1, tau_f1 = m._impl.def.frictionloss, m._impl.tendon_hasfrictionloss
    if (def_f1.any() or tau_f1.any()) and not {
        m.opt.disableFlags & DisableBit.FRICTIONLOSS
    }:
        # = d._impl.eff_frictionloss
        # = 1.0 / (d._impl.eff_D + (d._impl.eff_D == 0.0) * mujoco.m2cmass)
        linear_mrg = (ctx.lref < -r * f) * (f > 0)
        linear_pos = (ctx.lref > r * f) * (f > 0)
        active = active & ~linear_mrg & ~linear_pos
        # force = linear_mrg * f + linear_pos * -f
        # mass = linear_mrg * (-0.5 * r * f * f - f * ctx.lref)
        # cost = linear_pos * (-0.5 * r * f * f + f * ctx.lref)
        # mass_cost = fmass_cost.sum()

    if m.opt.cone == ConeType.FRICTIONCONE:
        # force = d._impl.eff_D * -ctx.lref * active + fmass_force
        cost = 0.5 * 3p.sum(d._impl.eff_D * ctx.lref * active)
        # m, h = 0.0, 0.0, 0.0
    elif m.opt.cone == ConeType.ELLIPSOID:
        # defcon = d._impl.contact.friction[d._impl.contact.dia > 1]
        # defcon = d._impl.contact.eff_addresses[d._impl.contact.dia > 1]
        # to prevent out of range append zeros to ctx.lref
        # else_m = 3p.zeros(
        #     lambda m: 3p.sum(dynamic_size(
        #         3p.concatenate((ctx.lref, 3p.zeros(10000)), (n,), (n,),
        #     ))
        # )
        # = solve_fn(eff_addresses) * ctx.lref
        # m, n, h = ctx.r(1, 0), m(1, 0), 3p.sum(mmh.norms)(def, 1, 1)
        # bottom cone: m(1, 0)
```

Carry Out : # 3264

polya how to solve it - Google | x | github.com mujoco - Google | x | [MJX] jax.lax.while_loop in st | x | [MJX] Use scan-based loop | x | +

github.com/google-deeppmind/mujoco/pull/3264

google-deeppmind / mujoco

Issues 154 Pull requests 181 Agents Discussions Actions Security and quality Insights

[MJX] Use scan-based loop in solver to enable reverse-mode autodiff (#2259) #3264

Open ingyukoh wants to merge 1 commit into google-deeppmind:main from ingyukoh:fix-mjx-solver-grad

Conversation 6 Commits 1 Checks 20 Files changed 4 +42

ingyukoh commented last week

Closes #2259.

Summary

Switch the outer constraint solver loop in `mjx/mujoco/mjx/_src/solver.py` from `jax.lax.while_loop` to the existing `_while_loop_scan` helper that the linesearch loop in the same file already uses. This unblocks `jax.grad` through `mjx.solve` for any user running with `m.opt.iterations > 1`.

Why

`jax.lax.while_loop` does not support reverse-mode AD:

```
ValueError: Reverse-mode differentiation does not work for lax.while_loop or lax.fori_loop with dynamic start/stop values. Try using lax.scan, or using fori_loop with static start/stop.
```

Until now the only workaround was `m.opt.iterations = 1`, which the issue reporter notes "leads to potentially inaccurate simulation and gradients."

What changed

One block at the bottom of `solve(...)`:

```
ctx = Context.create(m, d)
if m.opt.iterations == 1:
    ctx = body(ctx)
else:
    # was: ctx = jax.lax.while_loop(cond, body, ctx)
    ctx = _while_loop_scan(cond, body, ctx, m.opt.iterations)
```

`_while_loop_scan` is the existing helper at lines 239–253 used by the linesearch. It is a `jax.lax.scan` of length `max_iter` where iterations past the convergence condition become no-ops via `jax.lax.cond`. Forward semantics match `jax.lax.while_loop`; `m.opt.iterations` is a static Python int on `Option`, so the scan length is known at trace time.

Test

Reviewers: btaba

Still in progress? Convert to draft

Assignees: No one assigned

Labels: None yet

Projects: None yet

Milestone: No milestone

Development: Successfully merging this pull request may close these issues.

[MJX] jax.lax.while_loop in solver.py prevents co...

Notifications: Unsubscribe

You're receiving notifications because you authored the thread.

2 participants

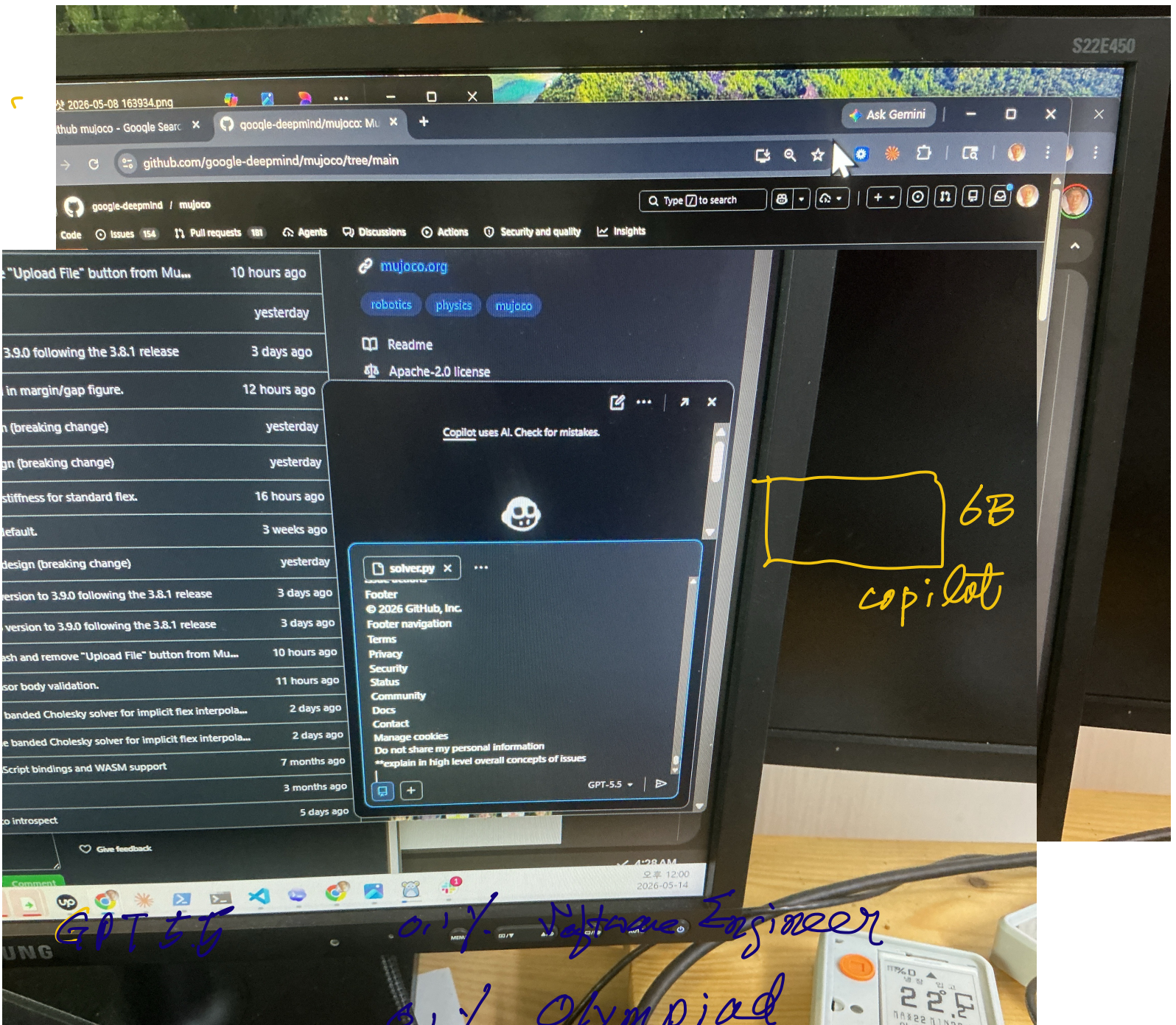
Allow edits by maintainers

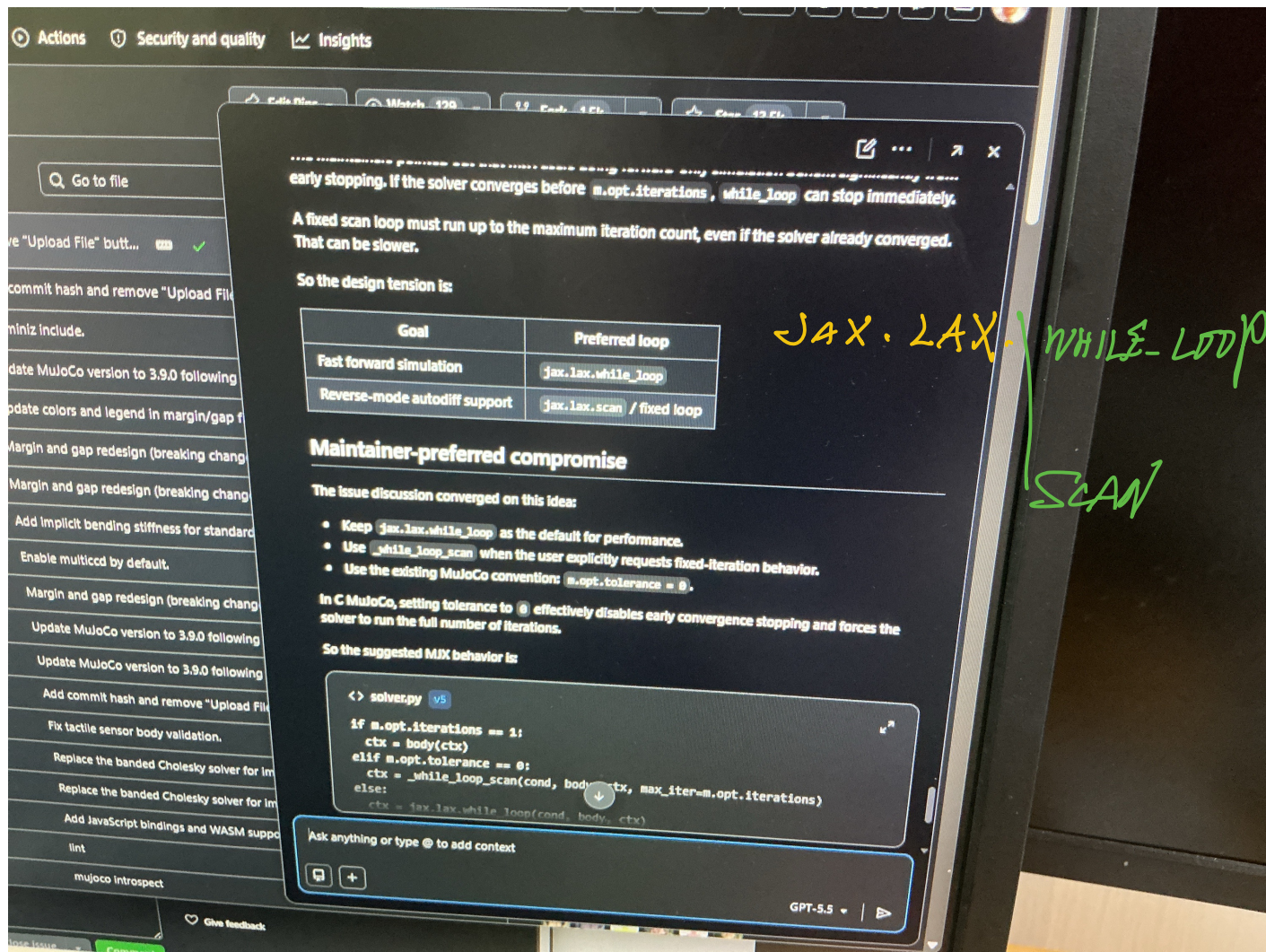
Back

tbata : reviewer

ask ||

solve





early stopping. If the solver converges before `m.opt.iterations`, `while_loop` can stop immediately.

A fixed scan loop must run up to the maximum iteration count, even if the solver already converged. That can be slower.

So the design tension is:

Goal	Preferred loop
Fast forward simulation	<code>jax.lax.while_loop</code>
Reverse-mode autodiff support	<code>jax.lax.scan</code> / fixed loop

Maintainer-preferred compromise

The issue discussion converged on this idea:

- Keep `jax.lax.while_loop` as the default for performance.
- Use `while_loop_scan` when the user explicitly requests fixed-iteration behavior.
- Use the existing MuJoCo convention: `m.opt.tolerance = 0`.

In C MuJoCo, setting tolerance to 0 effectively disables early convergence stopping and forces the solver to run the full number of iterations.

So the suggested MDX behavior is:

```
<> solver.py v5
if m.opt.iterations == 1:
    ctx = body(ctx)
elif m.opt.tolerance == 0:
    ctx = while_loop_scan(cond, body, ctx, max_iter=m.opt.iterations)
else:
    ctx = lax.lax.while_loop(rond, body, ctx)
```

Ask anything or type @ to add context

GPT-5.5